# § 4. Битовые множества и массовые операции

Когда элементы принадлежат небольшому известному универсуму, множество можно хранить как последовательность флагов. Машинное слово объединяет десятки таких флагов: одна операция OR или AND обрабатывает сразу несколько элементов. Выигрыш возникает не от изменения математического смысла задачи, а от упаковки однотипной работы.

После главы вы сможете реализовать динамический битсет на C++17, оценивать его стоимость по числу слов, применять сдвиги к достижимым суммам и строить битовые варианты BFS, транзитивного замыкания и исключения Гаусса над GF(2).

## 4.1. Интерфейс множества и размер универсума

Пусть $U = \{0, 1, \dots, m-1\}$. Нужны вставка, удаление, проверка принадлежности, объединение, пересечение и разность. Число элементов множества $k$ может быть намного меньше $m$. Эти параметры нельзя смешивать: битсет расходует память на весь универсум, даже если в нём содержится только один элемент.

Массив логических флагов уже даёт постоянный доступ по номеру. Упакуем флаги в слова ширины $w = 64$. Для элемента $x$ номер слова равен $\lfloor x/64 \rfloor$, номер бита — $x \bmod 64$. Всего нужно $M = \lceil m/64 \rceil$ слов.

Инвариант представления: бит $x \bmod 64$ слова $x/64$ равен единице тогда и только тогда, когда $x$ принадлежит множеству. Неиспользуемые старшие биты последнего слова всегда нулевые. Последнее условие особенно важно после сдвигов и дополнения: иначе подсчёт элементов может включить объекты вне универсума.




Рисунок 1: Адрес элемента распадается на номер слова и позицию бита; граница слова не является особым элементом множества.

Для учебной ширины $w = 8$ и $m = 20$ элемент 13 находится в слове 1, бите 5; элемент 16 — в слове 2, бите 0. Элемент 20 недопустим. В реальном коде ширина 64, но рассуждение остаётся тем же.

**Пример из видео.** В записи 28.09.2021 около 16:13 видны `Insert(x)` и `Test(x)` с номером слова и маской. Ключевой переход — от интерфейса множества к расположению бита. Значения 13 и 16 здесь выбраны для ясного показа границы, а не скопированы с доски.

## 4.2. Почему вставка — OR, а не сложение

Маска элемента — `std::uint64_t{1} << (x % 64)`. Вставка выполняет OR с маской, удаление — AND с инверсией маски, проверка — выделение соответствующего бита. Повторная вставка должна быть идемпотентной: множество не меняется, если элемент уже присутствует.


(c) Aleksey Tolstikov

Возьмём восьмибитное слово `00000100`, представляющее элемент 2. OR с той же маской оставит `00000100`. Арифметическое сложение даст `00001000`: элемент 2 исчез, появился элемент 3. Перенос разрушил смысл множества. Это не редкий граничный случай, а нарушение основного контракта.

Именно такой смысл обсуждается в коротком фрагменте около 15-й минуты записи 28.09.2021: сложение безопасно лишь при дополнительной гарантии, что бит ещё не установлен. В интерфейсе обычного множества такой гарантии нет, поэтому используем OR.

Для $A = \{0, 2, 5\}$ и $B = \{2, 3, 5\}$ маски равны `00100101` и `00101100`. OR даёт `00101101`, то есть $\{0, 2, 3, 5\}$; AND даёт `00100100`, то есть $\{2, 5\}$; XOR даёт $\{0, 3\}$. Порядок записи двоичного числа идёт от старшего бита слева к младшему справа: индекс элемента не равен позиции символа в строке.

| Операция | Одно слово | Всё множество |
|---|---|---|
| Вставка или удаление элемента | Одна маска и логическая операция | $O(1)$ |
| Проверка принадлежности | Сдвиг или AND | $O(1)$ |
| Объединение, пересечение, разность | OR, AND, AND-NOT | $O(M)$ |
| Сдвиг на заданное число позиций | Возможен перенос между словами | $O(M)$ |

Фраза «битовые операции работают за константу» относится к одному машинному слову. Множество на миллион возможных элементов занимает много слов, и целиком обработать его за одну инструкцию нельзя.

## 4.3. Безопасная реализация на C++17

Класс `Bits` разделён на несколько последовательных листингов; метод сдвига в пункте 4.4 завершает тот же класс. Полный исходник с заголовками — `code/04-bitset.hpp`.

Полный класс находится в `code/04-bitset.hpp`. Используем беззнаковый тип фиксированной ширины. В отличие от `1 << bit`, запись `std::uint64_t{1} << bit` сразу выбирает нужную ширину левого операнда. Сдвиг на 64 для 64-битного слова недопустим, поэтому все такие выражения требуют проверки.

```
class Bits {
    int n;
    std::vector<std::uint64_t> w;
    void trim() {
        if (n % 64) w.back() &= (std::uint64_t{1} << (n % 64)) - 1;
    }
public:
    explicit Bits(int size) : n(size), w((size + 63) / 64, 0) { assert(size >= 0); }
    int size() const { return n; }
    bool test(int x) const {
        assert(0 <= x && x < n);
        return (w[x/64] >> (x%64)) & 1;
    }
    void set(int x) {
        assert(0 <= x && x < n);
        w[x/64] |= std::uint64_t{1} << (x%64);
    }
    void reset(int x) {
        assert(0 <= x && x < n);
        w[x/64] &= ~(std::uint64_t{1} << (x%64));
    }
```

Операции над двумя множествами требуют одинаковых универсумов. Их нельзя молча объединять, сравнивая только общие слова: это скрыло бы ошибку контракта. Учебный класс использует `assert`; предусловия — $0 \le m \le 10^6$, допустимые индексы и неотрицательные сдвиги.

```
    void unite(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] |= b.w[i];
    }
    void intersect(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] &= b.w[i];
    }
    void subtract(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] &= ~b.w[i];
    }
    void xor_with(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] ^= b.w[i];
    }
    int count() const {
        int result=0;
        for (auto x:w) while (x) { x &= x-1; ++result; }
        return result;
    }
```

`count` использует тождество `x & (x-1)`: оно удаляет младший установленный бит. Поэтому внутренний цикл выполняется по числу единиц, а не по числу всех позиций. Для $M$ слов и $k$ единиц получаем $O(M + k)$ элементарных действий; при фиксированном $w = 64$ это также $O(M)$ с константой до 64. Это не обязательно одна аппаратная инструкция на слово.

В C++20 есть `std::popcount`, а в C++17 можно встретить расширение компилятора `__builtin_popcountll`. Здесь выбран переносимый вариант, чтобы контракт не зависел от расширения компилятора. Замена локальной функции подсчёта не меняет логику множества.

## 4.4. Сдвиг и достижимые суммы

Сдвиг на $d$ переводит каждый элемент $x$ в $x+d$, отбрасывая результаты вне универсума. Если $d = 64q+r$, то слово перемещается на $q$ слов, а при $r \neq 0$ делится между двумя соседними словами результата.

```
    Bits shifted(int d) const {
        assert(d >= 0);
        Bits b(n);
        if (d >= n) return b;
        int whole=d/64, part=d%64;
        for (int i=0; i<int(w.size()); ++i) {
            int j=i+whole;
            if (j<int(w.size())) b.w[j] |= w[i] << part;
            if (part && j+1<int(w.size()))
                b.w[j+1] |= w[i] >> (64-part);
        }
        b.trim();
        return b;
    }
};
```

При `part == 0` вторую запись пропускаем: выражение `>> (64-part)` тогда означало бы сдвиг на 64. При $d \geq m$ результат пуст. Последний вызов `trim` убирает биты за границей универсума. Исходный объект не меняется, поэтому сдвиг пригоден для перехода динамического программирования без смешивания старого и нового состояния.

Пусть бит $s$ означает, что сумма $s$ достижима некоторым подмножеством уже обработанных предметов. Изначально достижима только сумма 0. Для нового неотрицательного веса $d$ обновление имеет вид

```
Bits shifted = reachable.shifted(d);
reachable.unite(shifted);
```

Результат содержит либо старую сумму без нового предмета, либо старую сумму плюс его вес. Все новые биты вычислены из **старого** состояния, поэтому каждый предмет используется не более одного раза. Если обновлять обычный массив достижимости слева направо, один и тот же предмет мог бы использоваться повторно.

Для весов 3, 2, 3 и максимальной суммы 8 получаем состояния $\{0\}$, затем $\{0,3\}$, затем $\{0,2,3,5\}$ и наконец $\{0,2,3,5,6,8\}$. Два предмета веса 3 различны, поэтому сумма 6 допустима. Нулевой вес не меняет множество достижимых сумм. Отрицательные веса требуют другого диапазона индексов и здесь не поддерживаются.

Для $n$ предметов и суммы до $W$ время равно $O(n\lceil(W+1)/64\rceil)$, память — $O(\lceil(W+1)/64\rceil)$ слов, включая временный сдвиг с постоянным множителем. При огромном $W$ такой подход непригоден, даже если предметов мало: в главе 6 используем meet-in-the-middle.

В расшифровке видео соответствующий сюжет находится примерно на 33–38-й минутах. Числовая трасса выше — редакторское упражнение; она проверяет смысл сдвига независимо от ошибок ASR.

## 4.5. Поиск в ширину: целый слой как множество

В ориентированном невзвешенном графе обычный BFS строит слои $L_d = \{v : dist(s,v) = d\}$. Пусть `adj[v]` — битовое множество исходящих соседей. Тогда следующий слой равен объединению соседей текущего слоя за вычетом уже посещённых вершин.

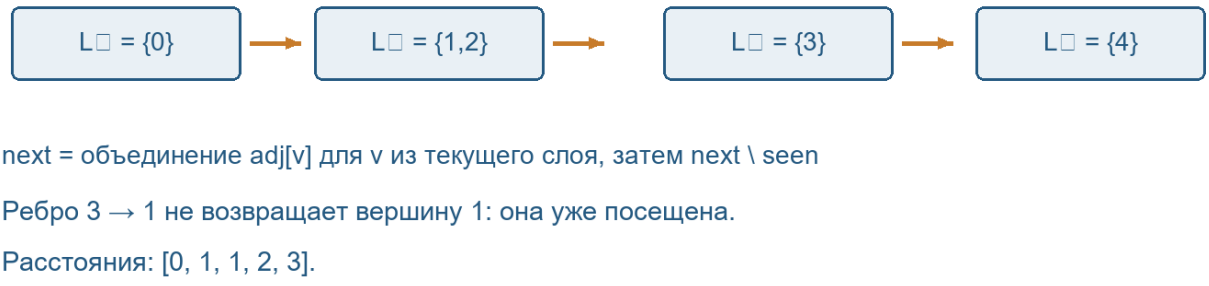



Рисунок 2: BFS собирает соседей всего слоя и исключает посещённые вершины до назначения новых расстояний.

Возьмём рёбра $0 \to 1, 0 \to 2, 1 \to 3, 2 \to 3, 3 \to 1, 3 \to 4$. Получаем $L_0 = \{0\}$, $L_1 = \{1, 2\}$, $L_2 = \{3\}$, $L_3 = \{4\}$. Обратное ребро $3 \to 1$ не возвращает вершину 1 в очередь: она уже посещена. Два пути к 3 также не создают двух копий вершины, потому что OR объединяет множества.

```
inline std::vector<int> bit_bfs(const std::vector<Bits>& adj, int s) {
    int n=int(adj.size());
    assert(0 <= s && s < n);
    for (const auto& row:adj) assert(row.size()==n);
    Bits seen(n);
    seen.set(s);
    std::vector<int> frontier{s}, dist(n,-1);
    dist[s]=0;
    int depth=0;
    while (!frontier.empty()) {
        Bits next(n);
        for (int v:frontier) next.unite(adj[v]);
        next.subtract(seen);
        seen.unite(next);
        frontier.clear();
        ++depth;
        for (int v=0; v<n; ++v)
            if (next.test(v)) { frontier.push_back(v); dist[v]=depth; }
    }
    return dist;
}
```

Инвариант: перед итерацией `frontier` содержит ровно вершины текущего слоя, а `seen` — все уже обнаруженные вершины до этого слоя включительно. Каждый новый сосед имеет путь длины на единицу больше; вершины с более коротким путём уже исключены. Поэтому первое присвоенное расстояние кратчайшее.

Важно честно посчитать реализацию. Строка каждой достижимой вершины объединяется один раз: $O(nM)$ словных операций в худшем случае. Здесь новый слой перечисляется простым просмотром всех $n$ позиций. Если число непустых слоёв равно $D$, это ещё $O(nD)$. Итого $O(nM + nD)$, а при длинной цепи остаётся $O(n^2)$. Упаковка не даёт автоматического ускорения любого графа.

Перечисление установленных битов по словам может сократить стоимость выделения новых вершин, но требует дополнительного аккуратного кода. Для разреженного графа обычный BFS за $O(n+e)$ часто лучше, а матрица битсетов расходует $O(nM)$ слов памяти. Этот вариант полезен прежде всего при плотных графах и массовых операциях над соседями.


(c) Aleksey Tolstikov

В видео около 48:40 видны слои BFS, посещённые вершины и граф. Сохраняем именно послойное объяснение; наш граф и код — новая проверяемая иллюстрация. Задача iRunner 3911 «Сумма расстояний» требует дополнительного чтения условия: это не просто вывод массива расстояний от одного источника.

## 4.6. Транзитивное замыкание

Теперь хотим ответить на все вопросы достижимости, не вычисляя длины путей. Пусть строка `reach[i]` содержит вершины, достижимые из $i$. На шаге $k$ разрешаем вершину $k$ как промежуточную: если $i$ уже достигает $k$, добавляем к строке $i$ всю строку $k$.

```
inline std::vector<Bits> closure(std::vector<Bits> reach) {
    int n=int(reach.size());
    for (int i=0; i<n; ++i) { assert(reach[i].size()==n); reach[i].set(i); }
    for (int k=0; k<n; ++k)
        for (int i=0; i<n; ++i)
            if (reach[i].test(k)) reach[i].unite(reach[k]);
    return reach;
}
```

В данной версии вершина достижима из себя по пути длины 0. Поэтому перед циклом ставим диагональные биты. Если задача требует только непустые пути, это соглашение надо изменить осознанно и отдельно проверить циклы.

Индуктивный инвариант алгоритма Уоршалла: перед шагом $k$ учтены пути, внутренние вершины которых лежат среди $0, \dots, k-1$. Новый путь либо уже существовал, либо распадается в $k$ на путь из $i$ в $k$ и из $k$ в $j$. Объединение строк реализует вторую возможность сразу для всех $j$.

Обновление на месте допустимо. На шаге $k$ строка $k$ объединяется только с самой собой и не меняется; изменение бита достижения $k$ в других строках не создаёт новой возможности, которой не было до шага. Стоимость — $O(n^2 M)$ словных операций и $O(nM)$ слов памяти. Это ускорение внутреннего цикла классического кубического алгоритма, а не переход к линейному времени.

## 4.7. Линейная алгебра над GF(2)

В поле GF(2) сложение и вычитание совпадают с XOR, а коэффициенты равны 0 или 1. Строку двоичной матрицы естественно хранить битсетом. Вычитание опорной строки из другой превращается в XOR целых слов.

```
inline int gf2_rank(std::vector<Bits> rows, int columns) {
    int rank=0, m=int(rows.size());
    for (const auto& row:rows) assert(row.size()==columns);
    for (int c=0; c<columns && rank<m; ++c) {
        int p=rank;
        while (p<m && !rows[p].test(c)) ++p;
        if (p==m) continue;
        std::swap(rows[p],rows[rank]);
        for (int i=rank+1; i<m; ++i)
            if (rows[i].test(c)) rows[i].xor_with(rows[rank]);
        ++rank;
    }
    return rank;
}
```


(c) Aleksey Tolstikov

После каждого найденного ведущего столбца одна новая строка становится опорной. Из нижних строк устраняется единица в этом столбце; прежние ведущие столбцы остаются нулевыми, поскольку опорная строка также имеет там нули. Число найденных ведущих позиций равно рангу.

Для строк `110`, `011`, `101` третья равна XOR первых двух, поэтому ранг равен 2. Независимость проверяется математическим равенством строк, а не количеством единиц. Порядок битов при печати надо согласовать с порядком столбцов: в примере строки записаны как обычные тройки коэффициентов слева направо.

Для системы $Ax = b$ добавляют последний бит правой части, но ведущие позиции ищут только среди столбцов переменных. Строка с нулевыми коэффициентами и правой частью 1 означает несовместность. Если противоречия нет, число свободных переменных равно числу переменных минус ранг; обратный ход восстанавливает решение после выбора свободных значений. Приведённая функция вычисляет только ранг коэффициентной матрицы и не выдаёт решение системы.

При $r$ строках и $c$ столбцах поиск ведущих строк занимает до $O(rc)$ проверок битов; исключение — $O(r \min(r, c)\lceil c/64 \rceil)$ словных операций. Логические операции сокращают стоимость обработки строки, но не отменяют поиск ведущих позиций.

## 4.8. Проверка, границы применимости и задания

Особенно полезны размеры универсума 0, 1, 63, 64, 65, 127 и 129. Они ловят переходы между словами и маскирование последнего слова. Для сдвига проверьте 0, 1, 63, 64, 65, $m$ и $m + 1$. Для каждой операции сравнивайте результат с массивом флагов; проверяйте все позиции, а не только `count`: одинаковое количество единиц не означает одинаковое множество.

BFS сравнивайте с обычной очередью на случайных маленьких графах; замыкание — с запуском обычного BFS из каждой вершины; достижимые суммы — с полным перебором подмножеств. Для ранга маленькой матрицы независимо посчитайте число разных XOR-комбинаций строк: оно равно $2^{rank}$. Это действительно иной способ проверки, а не переписывание того же исключения Гаусса.

**Практика.** Реализуйте множество и независимую модель на флагах, затем добавьте задачу достижимой суммы. BFS и GF(2) образуют углублённую часть недели. В следующей главе к вопросу выбора представления добавится порог: часть данных выгодно обрабатывать непосредственно, а часть — через заранее подготовленные сводки.

## 4.9. Практика к главе

В расширенной подборке приложения A: №3911 «Сумма расстояний» (основная); №4148 «Опасность» (резерв КР). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

## 4.10. Вопросы для самопроверки

1. Чем отличаются размер множества и размер универсума?
2. Где находятся элементы 63 и 64 в словном представлении?
3. Покажите ошибку повторной вставки через сложение.
4. Почему инверсия последнего слова требует внимания к лишним битам?
5. Объясните отдельный случай сдвига, кратного 64.
6. Почему временная копия сдвига предотвращает многократное использование предмета?
7. Зачем исключать посещённые вершины до формирования следующего слоя BFS?
8. Как вычислить вклад слоя BFS глубины d в сумму квадратов расстояний? Что добавить для недостижимых вершин по контракту задачи 3911?
9. Какое соглашение о достижимости вершины из себя принято в `closure`?
10. Чем ранг матрицы отличается от решения системы с правой частью?


(c) Aleksey Tolstikov